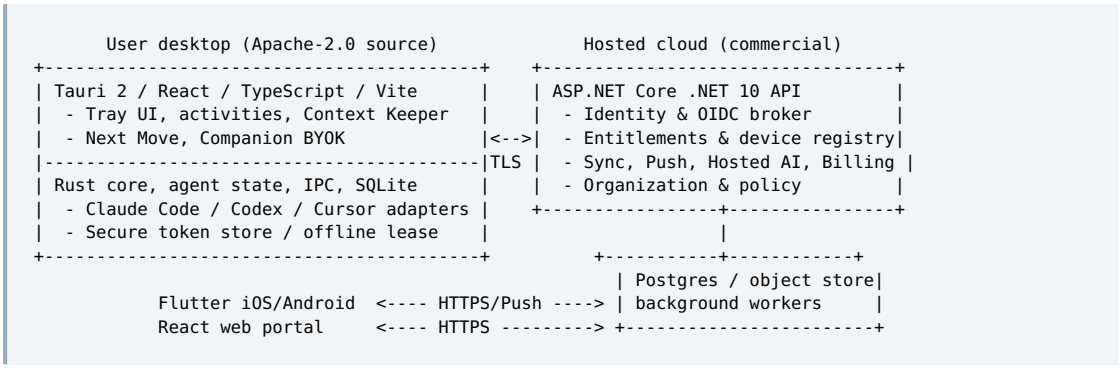
Lullshift - Technical Architecture and Stack

Version: 1.1 draft | **Date:** 2026-11-04 | **Audience:** Engineering and Claude Code implementation agents

1. Executive architecture

A **local-first Tauri 2 desktop application** communicates with agent adapters on the user's computer. The open-source client requires a **mandatory hosted Lullshift account** (email, Google, GitHub or Apple) to activate, but performs agent monitoring, activities, storage and BYOK conversations locally. A commercial ASP.NET Core backend provides identity for **all** editions and optional paid sync, notifications, hosted AI, billing and organization functions. A paid Flutter companion and React web console connect through hosted APIs.

Important architectural distinction: mandatory online initial authentication does not mean every local event is routed through the cloud. The client must operate during temporary outages with a valid offline lease; local workflows never require continuous network availability.



2. Stack decisions

Layer	Technology	Notes
Desktop runtime	Tauri 2	system tray, notifications, autostart, signed updates
Desktop UI	React + TypeScript + Vite	React Router, typed boundaries
Styling	Tailwind CSS + shadcn/ui	accessible primitives; no vendor lock-in
UI state	Zustand	local ephemeral state; Rust/SQLite authoritative
Rust crates	Rust stable, serde, tokio, sqlx/ rusqlite	choose one SQLite library after spike, use transactions
Local database	SQLite	versioned migrations and WAL; no plaintext secrets
Local agent IPC	Unix domain socket / Windows named pipe	authenticated local endpoint; optional restricted loopback API
Adapter packaging	Native hook forwarder + Rust adapter registry	explicit capability reporting
Local AI	Provider abstraction, BYOK HTTP clients	OS keychain via native API, no browser storage for secrets
Cloud API	ASP.NET Core / .NET 10	modular monolith, REST + SignalR
Cloud ORM/db	EF Core + PostgreSQL	migrations, tenant/org isolation
Auth	OpenIddict or managed OIDC broker	compare operational cost; provider federation
Mobile	Flutter	iOS + Android; commercial companion
Web console	React + TypeScript	account, linked identities, devices, subscription, org admin
Push	APNs + FCM	cloud-mediated notification delivery
Jobs	ASP.NET hosted workers + PostgreSQL queue	dedicated broker later if needed
Backups	S3-compatible object storage	client-side encryption for private content
Billing	Payment provider abstraction	provider chosen after regional policy review
Observability	OpenTelemetry	minimal, scrub PII and task content
Deploy	Docker on Linux + private registry	initial Compose, later orchestrator if justified

Use explicit version pins only after bootstrap resolves stable compatible releases. Keep documented framework constraints in repo `DECISIONS.md`.

3. Repository design

```
lullshift/
  CLAUDE.md
  docs/                          # PRD, architecture, ADRs
  .claude/agents/                # specialist delegations
  .claude/skills/                # workflow skills
  apps/desktop/                  # Tauri + React UI
  crates/agent-core/             # event model and state reducer
  crates/agent-adapters/         # hook adapter contracts
  crates/agent-ipc/              # native pipe/socket, auth
  crates/local-store/            # SQLite migrations/repository
  crates/activity-runtime/       # pause/resume, permissions
```

```
  packages/agent-protocol/       # versioned JSON schemas/types
  packages/activity-sdk/         # sandboxed activity contracts
  activities/{arcade,break,next-move,context-keeper,companion}/
  tests/fixtures/                # synthetic hooks, event replay
  sprints/                       # implementation backlog

lullshift-commercial/            # optional later separate repository
  backend/                       # .NET modules
  mobile/                        # Flutter
  web/                           # React
  infra/                         # Compose/deployment
```

The provided handoff is **documentation and Claude Code configuration**, not a preimplemented software repository.

4. Domain model and event protocol

4.1 Entities

- `Account` (cloud): UUID, verified email(s), profile, plan, consents, created/updated timestamps.
- `ExternalIdentity` (cloud): provider, subject ID, account ID, verified claims, link metadata. Unique `(provider, subject)`.
- `Device` (cloud): device UUID, account ID, platform, public key, last seen, revoked.
- `EntitlementLease` (signed): account, device, plan, issued/expiry, key ID, policy version; validate locally.
- `AgentSession` (local): UUID, adapter, provider session ID, workspace alias, state, observed timestamps, confidence.
- `AgentEvent` (local): event UUID, session ID, event type, sequence, provider timestamp, receipt timestamp, payload reference.
- `ContextBookmark` (local): session, intent, approved notes, review items, retention policy.
- `ActivityInstance` (local): activity ID, session link, lifecycle status, serializable state.
- `NextMoveItem` (local): session, content, status, created time.
- `CompanionThread` (local): thread and user-approved context scopes.
- `SyncItem` (local/cloud): opaque encrypted payload, vector/sequence, version and device ID.

4.2 Event schema (illustrative, version 1)

```
{
  "schemaVersion": 1,
  "eventId": "uuid",
  "provider": "claude-code",
  "providerSessionId": "opaque",
  "eventType": "needs_attention",
  "timestampUtc": "2026-11-04T10:00:00Z",
  "correlationId": "opaque",
  "capabilities": ["lifecycle", "permission_prompt"],
  "metadata": { "reason": "permission_request" }
}
```

Contract rules: preserve raw provider message locally only when needed/allowed; validate bounded input; dedupe by stable event ID; tolerate reordered callbacks; state changes are idempotent. A stop/turn-end event can be `idle` or `unknown`, not necessarily `completed`.

4.3 State machine

States: `unknown`, `idle`, `working`, `needs_attention`, `completed`, `failed`, `cancelled`. Explicit transitions with timestamps and source confidence; providers can emit only partial signals. Restore sessions as `unknown` after restart until confirmed. Stale timeout triggers `unknown` rather than a false success notice.

5. Authentication, identity, mandatory account

5.1 Boundary

Identity and plan are issued by Lullshift Cloud to every client, including Community. Account enrollment and first activation require Internet. After activation, the client validates a signed offline lease without querying the server for each event. **Proposed 30-day maximum offline lease**; product owner must ratify revocation/security tradeoff. No indefinite anonymous mode.

5.2 Login providers and email

- Email: verified passwordless magic link or OTP with one-time nonce, expiration, rate limiting and anti-enumeration; optionally add passkeys later.
- Google: OIDC federation with validated issuer/audience/nonce.

- GitHub: OAuth provider integration with verified email handling; do not depend on public GitHub email presence.
- Apple: Sign in with Apple and private relay handling; validate provider ID token and provider requirements.
- All providers map to one internal account through unique provider subject identifiers. Linking an identity requires a signed-in session and proof-of-control. Never merge accounts solely because two emails match.

5.3 Native OAuth/OIDC flow

1. Desktop creates state, nonce and PKCE verifier; invokes external system browser.
2. Browser reaches hosted authorization endpoint; user selects email, Google, GitHub or Apple.
3. Authorization code returns through **loopback redirect or registered custom URI scheme**; enforce redirect allowlist and ownership controls.
4. Native client exchanges code with PKCE; verify issuer, audience, nonce, state, clock skew and token signatures as applicable.
5. Persist refresh token only in OS secure storage; cache access token in memory, rotate refresh tokens and revoke devices on logout.
6. Call entitlement endpoint; verify signed offline lease using embedded public verification key and bound device identity.
7. On temporary outage, continue core local features while valid lease lasts. On expiry, show account-required reauthentication without deleting local data.

Browser redirects must not use embedded webviews. Public desktop/mobile clients cannot hold a confidential OAuth client secret. Align with RFC 8252 (https://www.rfc-editor.org/rfc/rfc8252).

5.4 Threat cases and controls

Threat	Control
Fake login callback / code substitution	state, nonce, PKCE, strict callback validation
Token theft from disk	OS keychain, short-lived access, rotation, revocation
Identity takeover through account linking	proof of current login and provider ownership, audit events

Threat	Control
Forged offline entitlement	asymmetric signature, device binding, expiry, key rotation
Rollback local system clock	detect anomalies, bounded lease, document limitations
Local malicious process injecting agent events	socket permissions, per-install secret/peer checks, validation, rate limits
Cloud server unavailable	lease-based local continuity and clear expiration UX
Cloud breach	minimize task data; optional client-side-encrypted sync
Organization-admin overreach	role scoping, no surveillance dashboards, tenant isolation

Clarify licensing: Apache-2.0 source grants rights to run/modify client; required account is a product policy in distributed official builds, not an attempt to restrict rights granted by Apache-2.0. A fork can remove the gate. State this honestly in commercial terms.

6. Local runtime and adapters

- Tauri holds tray and window; Rust core stays alive when window is hidden; shutdown explicitly releases hooks/IPC.
- Use authenticated UDS or Windows named pipes; no open unauthenticated TCP listener.
- Hook forwarder is a small packaged native executable invoked by supported agent tools, with timeouts, structured JSON and bounded logs.
- Existing agent configurations must be read, backed up and merged atomically, with uninstall restoration and user consent.
- Adapter capabilities include `canObserveLifecycle`, `canObserveAttention`, `canObserveCompletion`, `canProvideTaskSummary`, `canRemoteApprove` (default false).
- Gracefully degrade when a provider lacks support. Avoid UI/terminal scraping as a default integration strategy.
- Keep core protocol independent from Tauri UI so it can later become a local daemon.

7. Activities and extensibility

Lifecycle: `initialize -> start -> pause -> resume -> saveState -> dispose`, plus optional `restoreState`. Permission manifest for activity: storage namespace, notifications, optional user-approved context access, AI invocation, network access. Plugins must not receive raw

codebase contents by default. Initial release ships only trusted bundled activities; third-party executable plugins are disabled until an isolation strategy is proven (webview origin isolation / validated message bridge / capability sandbox). No dynamic unrestricted native plugins.

8. Context and local persistence

SQLite migrations with tests and backup/rollback plan; WAL and atomic writes. Store metadata-only agent sessions by default. Per-session user-created context and decisions are opt-in. Store OAuth credentials exclusively in OS secure store; store non-secret references in SQLite. Provide data export (JSON and Markdown), delete-all-local-data control and configurable retention.

9. Commercial cloud design

One modular ASP.NET Core .NET 10 deployment initially, with modules:

1. `Identity`: federation, email verification, OIDC endpoints, refresh/revocation.
2. `Accounts`: profiles, linked providers, consents, deletions, entitlements.
3. `Devices`: key registration, trusted devices, enrollment, revoke, signed offline leases.
4. `Subscriptions`: feature entitlements, payment events, invoices, fraud/abuse safeguards.
5. `Sync`: upload/download of versioned client-side encrypted records, conflicts and reconciliation.
6. `Push`: device tokens, notification priorities, dedupe and APNs/FCM delivery.
7. `HostedAI`: user-approved prompt boundary, capped usage, provider routing, audit/cost controls.
8. `Organizations`: membership, SSO/provisioning, policy and deployment preferences.

Use PostgreSQL + EF Core with explicit org/tenant IDs and authorization per resource. Prefer durable database jobs to introducing a broker in v1. Add object storage for backup blobs. Use HTTPS, rate limits, structured logging with redaction and OpenTelemetry. Web portal: React. Mobile: Flutter for paid features (status, notifications, notes, Companion); not a coding agent runner.

Example endpoints (draft)

```
GET    /v1/me
GET    /v1/identity/providers
POST   /v1/devices/enroll
POST   /v1/devices/{deviceId}/revoke
GET    /v1/entitlements/lease
POST   /v1/sync/batches
GET    /v1/sync/changes?cursor=...
POST   /v1/notifications/register-device
POST   /v1/notifications/relay
GET    /v1/subscriptions/current
POST   /v1/accounts/export
DELETE /v1/accounts/me
```

Endpoints and OIDC standards contracts must be refined in ADRs and OpenAPI before implementation. Never invent direct provider-token trust in the APIs.

10. Synchronization semantics

- Offline-first local writes; encrypted outbound queue with client-generated stable IDs.
- Cloud stores opaque ciphertext for sensitive content, not searchable plaintext.
- Sync preferences, explicit notes and activity state only when opted in; raw prompts and source code are excluded by default.
- Concurrent edits: scoped last-write-wins for harmless preferences; conflict-copy/manual resolution for notes, never silent deletion.
- Handle account switching and device revocation without silently merging local work from two identities.
- Prefer incremental cursor + idempotent upsert; append-only audit for critical entitlement/device changes.

11. Mobile security and push behavior

Flutter mobile signs in with external-browser PKCE, OS secure storage, provider broker. Mobile push carries only minimal opaque session reference and generic attention message by default. Detailed context is fetched after authenticated unlock. APNs/FCM token rotation supported. Remote approval is **out of scope** until separately threat modeled and adapter-authorized; never infer approval from tapping a notification.

12. Testing and operations

- Rust unit tests for reducer, dedupe, ordering, lease verification, IPC validation and migrations.
- TypeScript tests for UI component behavior and activity lifecycle; end-to-end smoke tests on every OS.
- Cloud integration tests for email verification, provider linking, PKCE, tokens, device revoke, plan changes and tenant isolation.
- Fixture-driven adapter tests for exact provider hook payloads, with unknown-state and restarts.
- Flutter tests for notification routing, background-to-foreground and device registration.
- CI: Rust fmt/clippy/test, TypeScript lint/typecheck/test/build, .NET restore/build/test, secret/dependency scan, SBOM, signing, artifact provenance.
- Deploy: Docker Compose Linux for cloud; dev uses fake email/ push providers. Ship signed updates; Tauri updater requires signatures (https://v2.tauri.app/plugin/updater/).
- Security review before enabling third-party activities, account linking, or remote actions.

13. Sequence and ADRs

1. Bootstrap repo and contract schemas; CI/tests.
2. Build mandatory auth / provider federation / device lease service and desktop sign-in.
3. Tauri tray + SQLite + Rust session state engine.

4. Claude Code hook forwarder and reversible installation.
5. Notifications, Break, Arcade.
6. Context Keeper, Next Move.
7. Companion BYOK and extension SDK.
8. Additional adapters and stability release.
9. Optional commercial sync/cloud and Flutter mobile.

Required ADRs: `ADR-001-tauri-react-rust`, `ADR-002-required-account-and-offline-lease`, `ADR-003-oauth-broker-and-providers`, `ADR-004-agent-event-contract`, `ADR-005-local-ipc`, `ADR-006-activity-sandbox`, `ADR-007-encrypted-sync`, `ADR-008-business-monetization`.

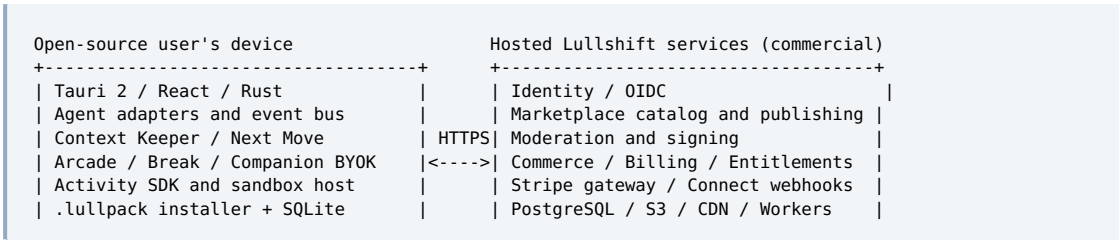
14. External reference URLs

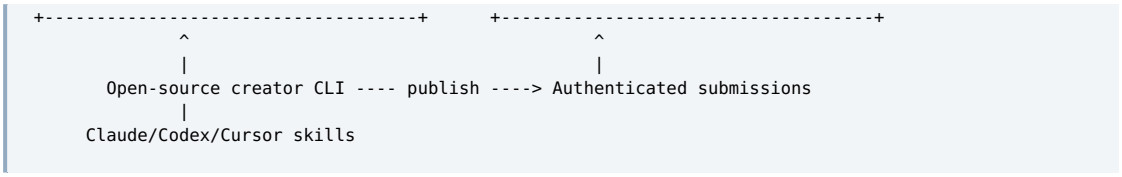
- Tauri 2: https://v2.tauri.app/
- Tauri updater: https://v2.tauri.app/plugin/updater/
- OAuth native apps: https://www.rfc-editor.org/rfc/rfc8252
- Claude Code skills and subagents: https://claude.com/blog/steering-claude-code-skills-hooks-rules-subagents-and-more
- Flutter: https://docs.flutter.dev/
- OpenIddict: https://documentation.openiddict.com/
- ASP.NET Core: https://learn.microsoft.com/aspnet/core/

Status: Design baseline. Provider support and exact dependency versions must be validated against current official documentation during each sprint.

Architecture Revision 1.1: Activity Packages, Creator Toolchain, Marketplace and Stripe

Component boundaries and ownership





The commercial backend remains an ASP.NET Core .NET 10 modular monolith with PostgreSQL and S3-compatible package storage. The mobile companion remains Flutter and uses the hosted account; mobile digital-goods checkout is excluded from the initial release.

Proposed `.lullpack` v1 format

A deterministic ZIP-compatible package with validated size and file limits, canonical manifest, assets and optional puzzle/content files. Reject `../`, absolute paths, links, nested archives, zip bombs, duplicate canonical filenames, executable native files, dangerous content types and unknown privileges. Use SHA-256 for package integrity and separate verifiable platform/publisher signatures. Reproducible builds should yield stable hashes for identical inputs.

```
{
  "schemaVersion": 1,
  "id": "com.example.sudoku",
  "version": "1.0.0",
  "name": "Sudoku",
  "category": "arcade",
  "entrypoint": "index.html",
  "runtime": "isolated-web-v1",
  "minHostVersion": "0.3.0",
  "sdkVersion": "1.0.0",
  "permissions": ["activity.storage"],
  "interruptibility": { "pause": true, "resume": true, "persistState": true },
  "offline": true
}
```

Host-verified metadata (publisher IDs, package digests, signing keys, review status and entitlement) lives outside the author-controlled manifest. Never trust an unsigned manifest's publisher or price claim. Schema is specified at `reference/ACTIVITY_MANIFEST_V1.schema.json`.

Activity sandbox and host bridge

Treat all downloaded/AI-generated activity JavaScript as attacker-controlled. Load into a dedicated isolated origin or WebView with no Tauri IPC access. Enforce CSP, trusted local asset URL handling, no remote navigation, strict default network-denial, no system clipboard without grant, no filesystem, no arbitrary host API, no shell and no

agent token exposure. Expose only a versioned, origin-bound, nonce-validated bridge for pause/resume and namespaced storage. OS/WebView sandbox guarantees vary and must be verified on all three desktop targets; if insufficient, block third-party package loading rather than shipping with a cosmetic sandbox.

The host controls interruption priorities. `needs_attention` must preempt activities without relying on plugin cooperation; persist last safely committed snapshot and allow recovery after crash. Bound CPU, memory and storage quota; enforce quotas and timeout/crash handling. Sensitive agent context is not a default permission; a separately reviewed user-granted capability may be considered in a future version.

Creator tooling and provider-independent AI prompts

Create open-source TypeScript SDK (`packages/activity-sdk`), build/validator CLI (`tools/lullshift-cli`), deterministic event simulator and starter templates. Claude Code-specific creator skill in `.claude/skills/activity-creator/SKILL.md`; generic creator prompt and alternative-tool wrappers in `creator/`. Creator builds and local validation must work without cloud services; uploading requires Lullshift sign-in. `create/dev/simulate/validate/pack/publish` are proposed CLI commands, not existing commands yet.

Pipeline: scaffold -> run local dev preview -> simulate interruption -> static/dependency analysis -> validate manifest and licensing -> deterministic pack -> authenticate -> upload to quarantine bucket -> server-side verification -> moderation -> sign release -> catalog and CDN. No package serves to users until approved. Perform published version immutability checks and handle emergency security revocation.

Marketplace modules

`Catalog`, `Publishers`, `Submissions`, `Moderation`, `PackageSigning`, `Entitlements`, `Orders`, `Payments`, `Subscriptions`, `CreatorSettlement`, `Reconciliation`. Include explicit authorization boundaries and auditable state transitions.

Suggested entities: `Publisher`, `SellerVerification`, `Activity`, `ActivityVersion`, `PackageArtifact`, `Listing`, `Price`, `Submission`, `ModerationCase`, `Order`, `OrderItem`, `Payment`, `PaymentEvent`, `Refund`, `Dispute`, `Entitlement`, `Commission`, `SellerTransfer`, `Payout`, `Subscription`, `InvoiceReference`. Use PostgreSQL outbox/inbox tables for durable idempotent webhook and fulfillment jobs, plus scoped object-storage access and content hashing.

Public API examples (tentative contracts, ADR required):

```
GET    /api/marketplace/activities
GET    /api/marketplace/activities/{id}/versions
POST   /api/creator/submissions
POST   /api/creator/submissions/{id}/finalize
GET    /api/library/entitlements
POST   /api/commerce/checkouts
POST   /api/webhooks/stripe
POST   /api/creator/connect/onboarding
GET    /api/creator/settlements
```

Permission model is seller-scoped; no creator can mutate another publisher's submissions or prices. Account links must never be inferred from unverified email aliases.

Payment architecture and financial invariants

The operator is **expected to incorporate outside Türkiye**, but legal jurisdiction is pending. Stripe Billing, Payments/Checkout and Connect are the preferred first integrations **conditional on official eligibility**. Implement `IPaymentCheckout`, `ISubscriptionBilling`, `IMarketplaceSellerOnboarding`, `ISettlementProvider`, `IPaymentEventVerifier` behind a provider-neutral domain. Avoid a fake generic interface that silently assumes all processors support destination charges, multi-party transfers, subscriptions and tax services.

- Use server-created Checkout Sessions and external browser checkout; never store card PAN/CVV or persist webhook signing secrets in logs.
- Store monetary values as integer minor units and ISO currency; validate price snapshots, tax policies, seller/MoR model, rounding and commission bases with finance review.
- `PaymentProviderEvent` keyed by provider+providerEventId. Verify signature using raw request body and current secret; process

asynchronously through transactional inbox/outbox with retries and idempotent handlers.

- Grant paid license only when trusted settled/paid status is confirmed; success redirect is insufficient. Refund and dispute states are audited and can suspend future downloads according to policy; retain clear rules for offline installed content.
- Stripe Connect: select destination charges vs separate charges/transfers **after** merchant-of-record and multi-seller basket decisions. Support platform fees and reversals without inventing settlement semantics.
- Leave payouts to regulated provider, require publisher KYC/KYB, country eligibility and payout account validation. Reconcile provider balance transactions, internal commission ledger, refund transfers and periodic finance reports.
- Limit offline entitlement duration and sign license leases using rotating asymmetric keys; time skew, revoked packages, refunds and reactivation require documented resolution. Mandatory initial login remains in place for Community.
- All provider integrations ship with fake providers, deterministic webhook fixtures, idempotency tests and real sandbox tests before enabling production.

Critical sequence sketches

Purchase: User -> Commerce creates frozen Order -> Stripe Checkout -> signature-verified provider event -> inbox processing -> paid Order -> durable Entitlement -> device obtains signed activity license -> installer checks package integrity/signature -> activity runs in sandbox.

Publisher: User -> account + verification -> Connect onboarding -> submit immutable version -> quarantine -> scanner + simulated sandbox + review -> package signing -> CDN publish -> later purchase -> provider settles seller amount and platform fee -> reconciled records.

Subscription: Account -> Billing checkout -> verified paid subscription event -> Pro/Business entitlement -> sync/mobile/ managed services. The Community client remains local-capable on temporary disconnection after activation.

ADRs required before launch

- ADR-020: Legal merchant-of-record, operator jurisdiction, tax registration, invoice obligations.
- ADR-021: Stripe Connect charge/transfer architecture and creator-country whitelist.
- ADR-022: Package sandbox security model per OS and restricted host bridge.
- ADR-023: Immutable package signing, key rotation, revocation and asset CDN.
- ADR-024: Offline licenses, refunds/disputes and revocation behavior.
- ADR-025: Creator ownership attestations, AI disclosure, licensing, moderation and appeals.

Engineering quality gates

Adversarial file/path tests; CSP/bridge isolation and cross-origin tests; abuse and upload throttling; deterministic pack reproducibility; plugin pause/resume/crash tests; package update/migration rollback; signed package tampering and compromised key scenarios; payment webhook signature and replay checks; concurrent duplicate orders and idempotent fulfillment; cross-account entitlement isolation; refunds/disputes and reconciliation; creator payout eligibility; audit-log redaction; mobile store rule review before any in-app sale.